

PROJECT DOCUMENTATION

Multi-Site Active Directory

with Zabbix Monitoring

Field	Details
Author	Diaa
Project	Multi-site AD (diaa.local) + Zabbix 7.0 monitoring
Version	1.0
Date	3 October 2026
Environment	Microsoft Azure (provisioned with Terraform; access through Azure Bastion)
Status	Completed - lab torn down after documentation

Table of Contents

1. Project Overview	3
1.1 Objective.....	3
1.2 Scope	3
1.3 Architecture Diagram	3
1.4 Server & IP Inventory.....	4
2. Active Directory	5
2.1 Forest & Domain Design.....	5
2.2 Domain Controllers.....	5
2.3 DNS & Delegation	6
2.4 Sites and Services	6
2.5 FSMO Roles.....	8
2.6 OUs, Groups and Group Policy	9
2.7 Additional Evidence	11
a) Replication health.....	11
b) RODC configuration.....	12
3. Zabbix Monitoring	Error! Bookmark not defined.
3.1 Zabbix Server Overview.....	13
3.2 Zabbix Agent 2 Configuration.....	14
3.3 Service Account (gMSA) & Access	15
3.4 Items	16
3.5 Discovery Rules.....	19
3.6 Triggers	20
3.7 Telegram Notifications	21
3.8 Dashboard	23
3.9 LDAP Authentication & JIT Users.....	24
3.9.1 LDAP Bind Account vs gMSA.....	26
3.10 Logs.....	28
4. Testing & Validation	29
5. Troubleshooting Log.....	30
6. Conclusion & Lessons Learned	31

1. Project Overview

1.1 Objective

The goal of this project was to build a multi-site Active Directory forest and monitor it end to end. The forest root domain diaa.local has two child domains (cairo.diaa.local and giza.diaa.local), an additional domain controller, a read-only domain controller (RODC), DNS delegation to the child domains and site-aware replication across four AD sites. The whole environment runs in Azure and was provisioned with Terraform. Zabbix 7.0 monitors every domain controller through Zabbix Agent 2, authenticates its users against Active Directory over LDAP with just-in-time (JIT) provisioning, and sends alerts to a Telegram group.

1.2 Scope

- Infrastructure: one Azure VNet with a subnet per server and Azure Bastion for administrative access, all deployed with Terraform
- Active Directory: PDC, additional DC, RODC, Cairo child domain, Giza child domain
- DNS delegation, AD Sites and Services, replication between sites
- Zabbix: agent monitoring of DCs, gMSA service account, LDAP login with JIT provisioning, triggers and Telegram alerts
- Out of scope: Exchange, AD FS, backup and disaster recovery, and PKI / LDAPS

1.3 Architecture Diagram

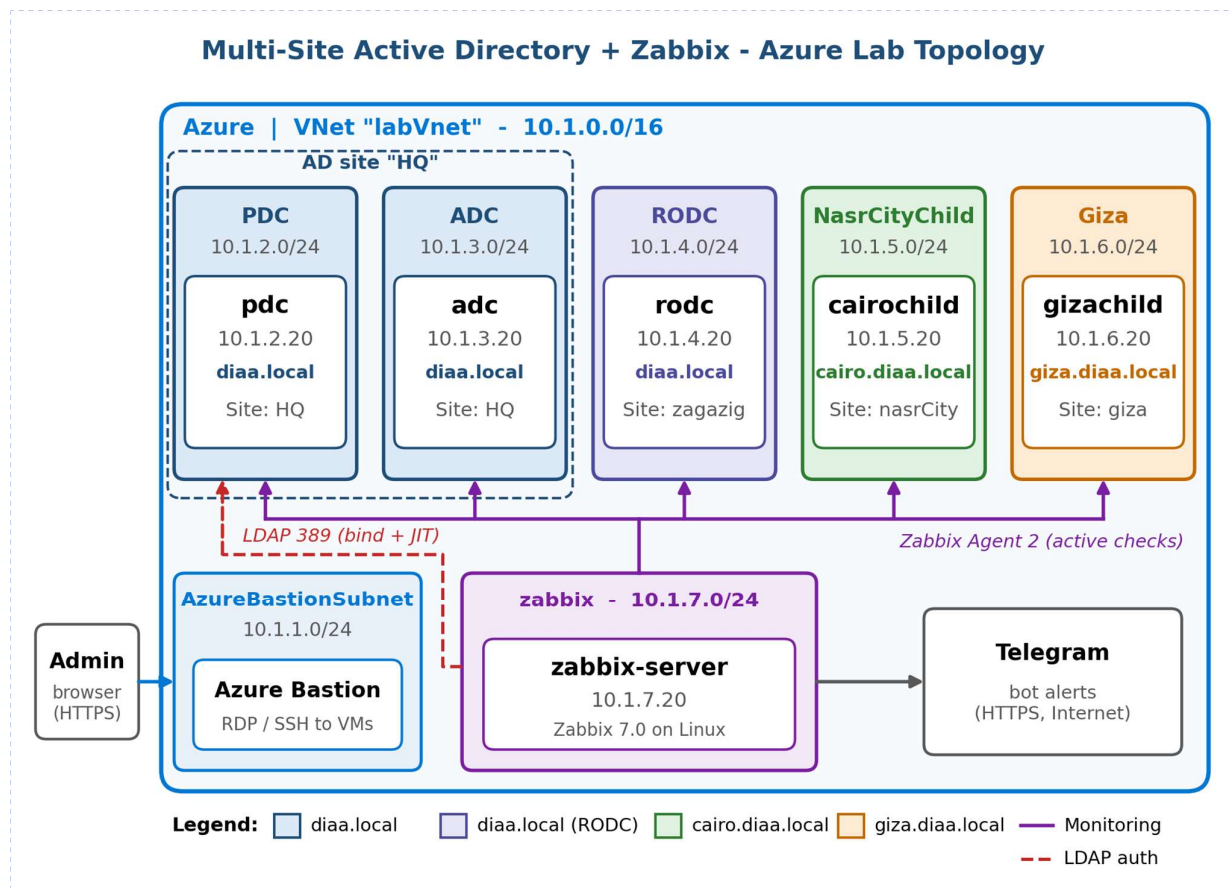


Figure 1: Azure lab topology - subnets, AD sites, domains and monitoring flow

All servers sit in one Azure VNet, labVnet (10.1.0.0/16), with one /24 subnet per server. Private IP addresses are static (the .20 address of each subnet), which keeps the DNS records, AD site subnets and Zabbix host interfaces stable.

Subnet	Address prefix	Contents
AzureBastionSubnet	10.1.1.0/24	Azure Bastion - browser-based RDP/SSH access to all VMs
PDC	10.1.2.0/24	pdcc (10.1.2.20)
ADC	10.1.3.0/24	adcc (10.1.3.20)
RODC	10.1.4.0/24	rodcc (10.1.4.20)
NasrCityChild	10.1.5.0/24	cairochild (10.1.5.20)
Giza	10.1.6.0/24	gizachild (10.1.6.20)
zabbix	10.1.7.0/24	zabbix-server (10.1.7.20)

1.4 Server & IP Inventory

Host name	Role	Domain	AD site	IP address	OS
pdcc	Primary DC, Global Catalog, FSMO holder	diaa.local	HQ	10.1.2.20	Windows Server 2022
adcc	Additional DC, Global Catalog, Infrastructure Master	diaa.local	HQ	10.1.3.20	Windows Server 2022
rodcc	Read-only DC, Global Catalog	diaa.local	zagazig	10.1.4.20	Windows Server 2022
cairochild	Cairo child domain DC	cairo.diaa.local	nasrCity	10.1.5.20	Windows Server 2022
gizachild	Giza child domain DC	giza.diaa.local	giza	10.1.6.20	Windows Server 2022
zabbix-server	Zabbix 7.0 monitoring server	-	-	10.1.7.20	Linux

2. Active Directory

2.1 Forest & Domain Design

Description

The forest root domain diaa.local was created on the PDC and two child domains, cairo.diaa.local and giza.diaa.local, were added to it, giving one forest with three domains. Every domain controller is also a Global Catalog server, so logons and forest-wide searches are served locally in each site.

Item	Value
Forest root domain	diaa.local
Child domains	cairo.diaa.local, giza.diaa.local
Forest / domain functional level	Windows Server 2016
Global Catalog servers	All five DCs: pdc, adc, rodc, cairochild, gizachild

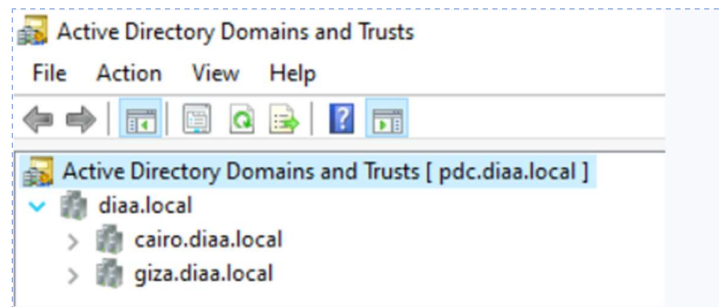


Figure 2: Active Directory Domains and Trusts / forest structure

2.2 Domain Controllers

Description

Each domain has its own domain controllers, shown here in Active Directory Users and Computers: pdc, adc and rodc in diaa.local, cairochild in cairo.diaa.local and gizachild in giza.diaa.local. The DC Type and Site columns confirm that every DC is a Global Catalog, that rodc is read-only, and which AD site each DC belongs to.

Name	Type	DC Type	Site	Description
adc	Computer	GC	HQ	
pdc	Computer	GC	HQ	
RODC	Computer	Read-only, GC	zagazig	

Figure 3: Domain controllers of diaa.local (pdc, adc, rodc)

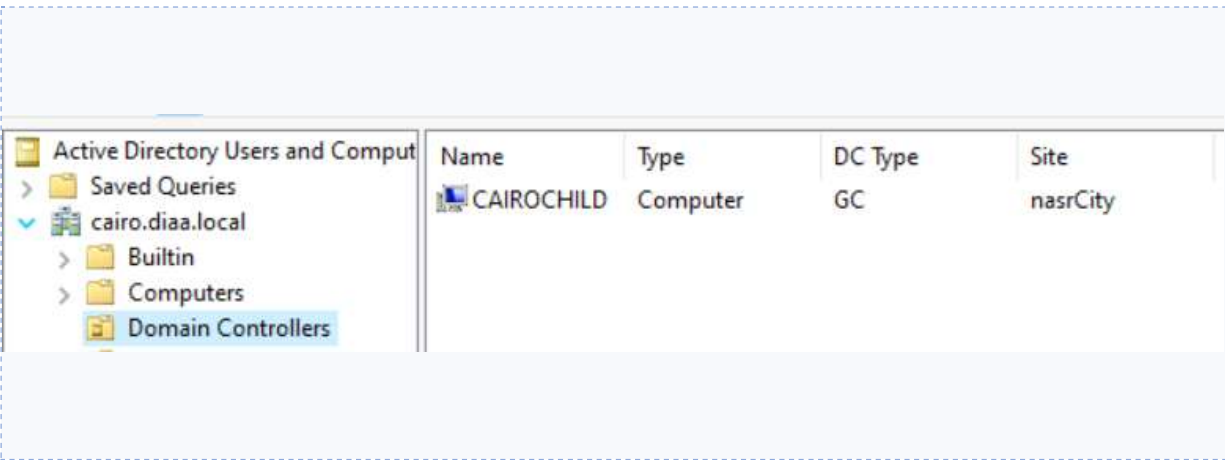


Figure 4: Cairo child domain controller

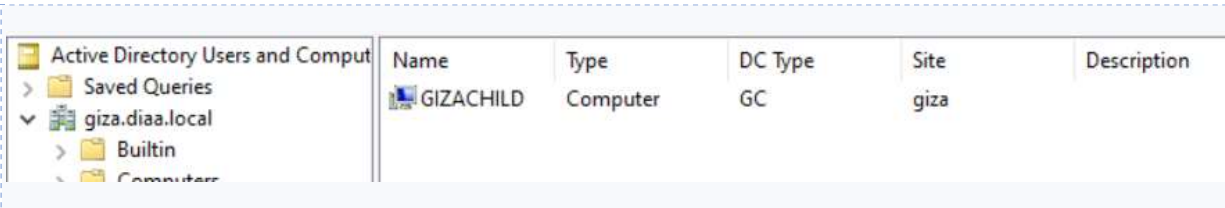


Figure 5: Giza child domain controller

2.3 DNS & Delegation

Description

The diaa.local DNS zone is hosted on the root domain controllers, and the cairo and giza child zones are delegated to the DCs of their own domains. Each child domain therefore resolves its names locally, while every client can still find all domains through the root. The DNS console also shows the _msdcs, _sites, _tcp and _udp service-record zones that Active Directory creates.

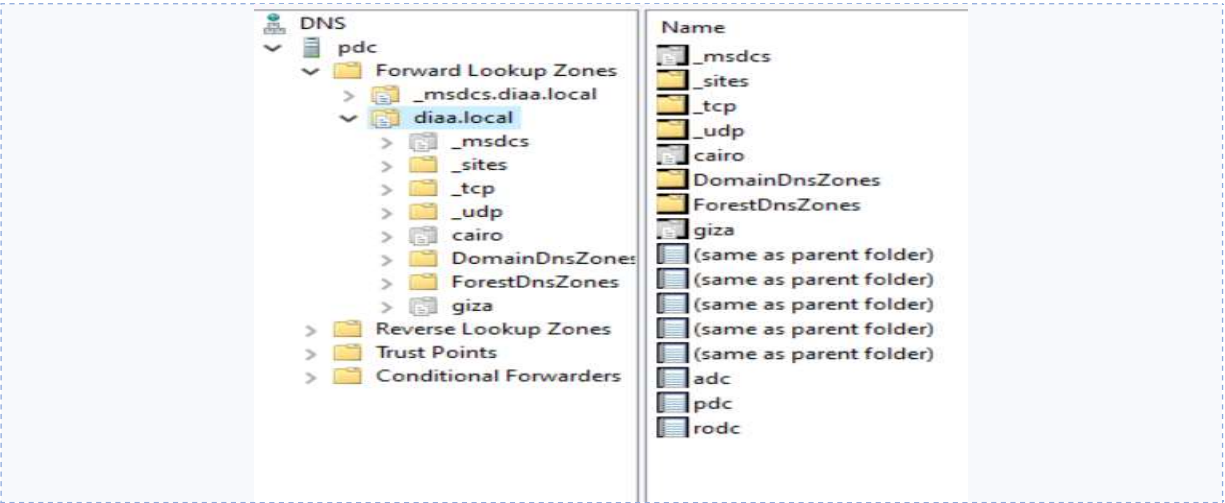


Figure 6: DNS zones and delegation for child domains

2.4 Sites and Services

Description

AD sites describe the network topology so that clients and replication use the closest, cheapest path. HQ holds pdc and adc, zagazig holds rodc, nasrCity holds cairochild and giza holds gizachild. Each Azure subnet is

mapped to its site, and site links from HQ to each other site (cost 100, replication every 15 minutes) control inter-site replication.

Site	Subnet	Domain controllers
HQ	10.1.2.0/24, 10.1.3.0/24	pdc, adc
nasrCity	10.1.5.0/24	cairochild
giza	10.1.6.0/24	gizachild
zagazig	10.1.4.0/24	rodc

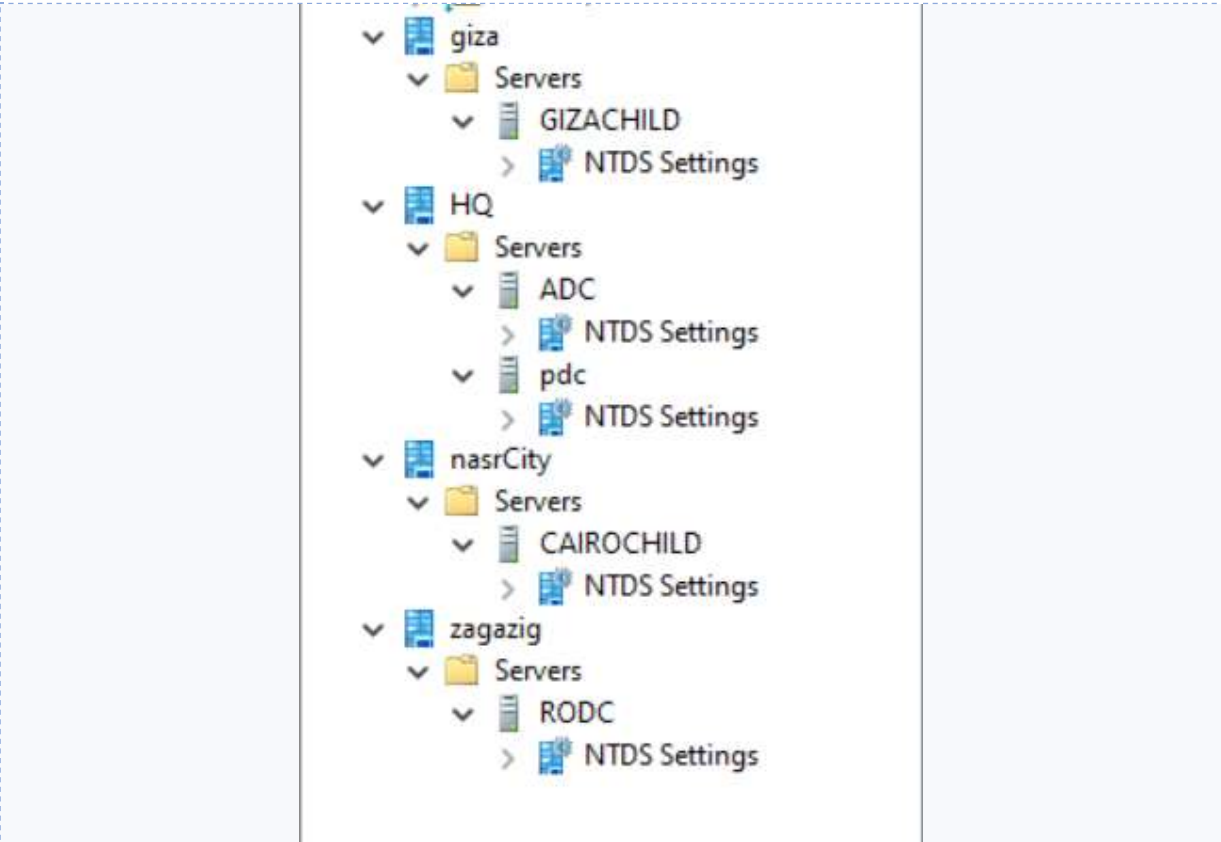


Figure 7: AD Sites and Services — sites and servers

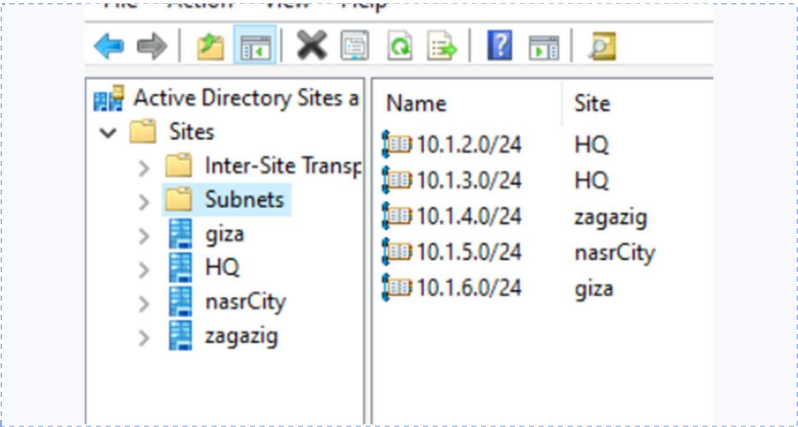


Figure 8: Subnets mapped to sites

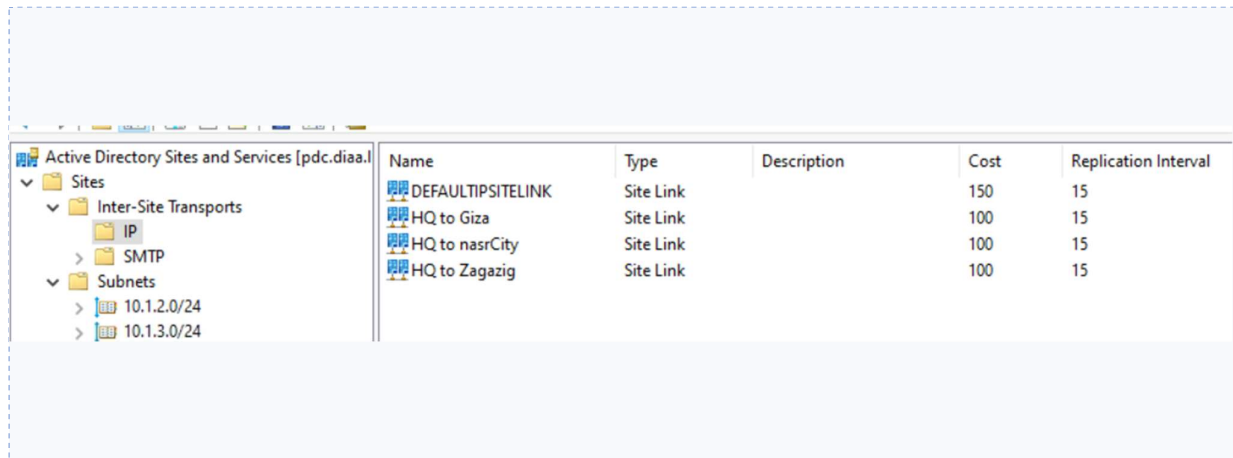
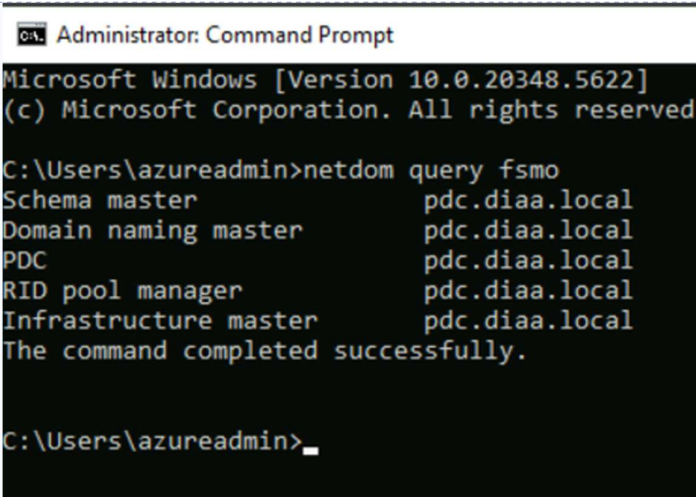


Figure 9: Site links and replication interval

2.5 FSMO Roles

Five roles exist: two are forest-wide (one holder in the whole forest) and three are domain-wide (one holder per domain). The RODC cannot hold any role. Roles are split in the root domain so the PDC does not carry everything: the Infrastructure Master was moved to ADC. Normally the Infrastructure Master should not sit on a Global Catalog server in a multi-domain forest, but because every DC in this forest is a Global Catalog the placement has no functional impact.

Domain	Role	Scope	Holder
diaa.local	Schema Master	Forest	pdcd
diaa.local	Domain Naming Master	Forest	pdcd
diaa.local	PDC Emulator	Domain	pdcd
diaa.local	RID Master	Domain	pdcd
diaa.local	Infrastructure Master	Domain	adc
cairo.diaa.local	PDC Emulator / RID / Infrastructure	Domain	cairochild
giza.diaa.local	PDC Emulator / RID / Infrastructure	Domain	gizachild
-	All roles	-	rodc: none (not supported)



```

Administrator: Command Prompt

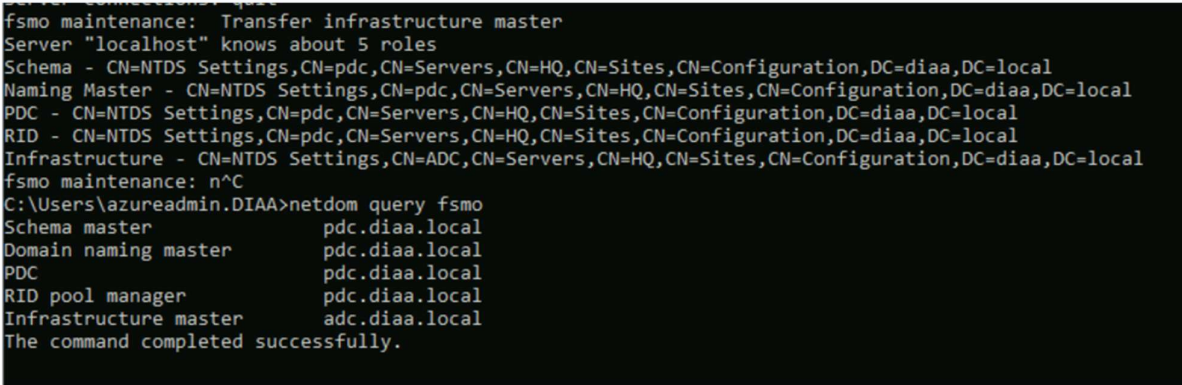
Microsoft Windows [Version 10.0.20348.5622]
(c) Microsoft Corporation. All rights reserved

C:\Users\azureadmin>netdom query fsmo
Schema master                pdc.diaa.local
Domain naming master         pdc.diaa.local
PDC                           pdc.diaa.local
RID pool manager             pdc.diaa.local
Infrastructure master         pdc.diaa.local
The command completed successfully.

C:\Users\azureadmin>

```

Figure 10: netdom query fsmo — BEFORE the role move

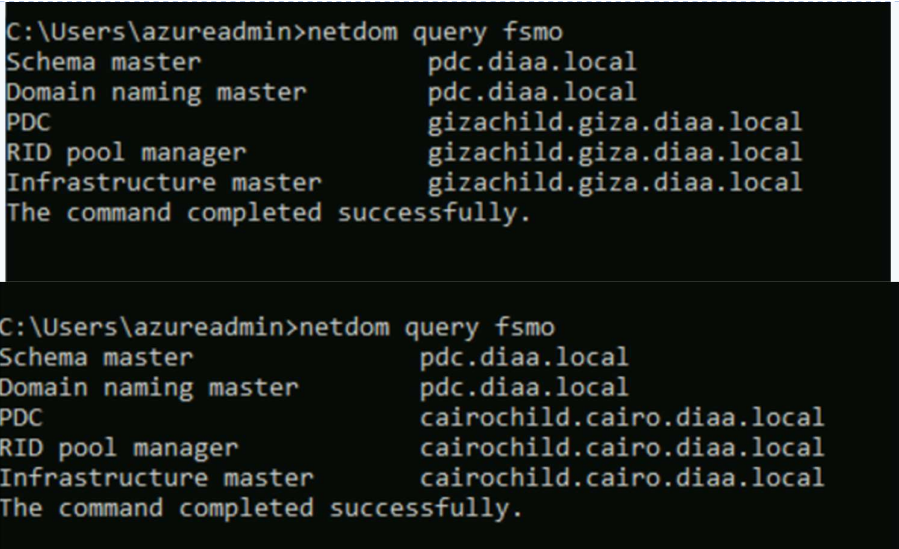


```

fsmo maintenance: Transfer infrastructure master
Server "localhost" knows about 5 roles
Schema - CN=NTDS Settings,CN=pdc,CN=Servers,CN=HQ,CN=Sites,CN=Configuration,DC=diaa,DC=local
Naming Master - CN=NTDS Settings,CN=pdc,CN=Servers,CN=HQ,CN=Sites,CN=Configuration,DC=diaa,DC=local
PDC - CN=NTDS Settings,CN=pdc,CN=Servers,CN=HQ,CN=Sites,CN=Configuration,DC=diaa,DC=local
RID - CN=NTDS Settings,CN=pdc,CN=Servers,CN=HQ,CN=Sites,CN=Configuration,DC=diaa,DC=local
Infrastructure - CN=NTDS Settings,CN=ADC,CN=Servers,CN=HQ,CN=Sites,CN=Configuration,DC=diaa,DC=local
fsmo maintenance: n^C
C:\Users\azureadmin.DIAA>netdom query fsmo
Schema master                pdc.diaa.local
Domain naming master         pdc.diaa.local
PDC                           pdc.diaa.local
RID pool manager             pdc.diaa.local
Infrastructure master         adc.diaa.local
The command completed successfully.

```

Figure 11: netdom query fsmo — AFTER the role move



```

C:\Users\azureadmin>netdom query fsmo
Schema master                pdc.diaa.local
Domain naming master         pdc.diaa.local
PDC                           gizachild.giza.diaa.local
RID pool manager             gizachild.giza.diaa.local
Infrastructure master         gizachild.giza.diaa.local
The command completed successfully.

C:\Users\azureadmin>netdom query fsmo
Schema master                pdc.diaa.local
Domain naming master         pdc.diaa.local
PDC                           cairochild.cairo.diaa.local
RID pool manager             cairochild.cairo.diaa.local
Infrastructure master         cairochild.cairo.diaa.local
The command completed successfully.

```

Figure 12: netdom query fsmo in the Cairo and Giza child domains

2.6 OUs, Groups and Group Policy

Rather than a generic OU tree, the directory objects here exist for a specific purpose: scoping and authorising Zabbix.

Object	Name	Purpose
OU	zabbixUsers	LDAP search scope: the Base DN in Zabbix points here, so only accounts in this OU can sign in to Zabbix
OU	zabbixSearch	Holds the LDAP bind (service) account, kept apart from the users it searches
Security groups	zabbixUsers / zabbixGuests	Matched by Zabbix through memberOf: zabbixUsers maps to Zabbix administrators (Super admin role), zabbixGuests maps to Guests (Guest role)
Test user 1	zab	Member of zabbixUsers: proves LDAP login and JIT provisioning with the admin role
Test user 2	zab-gust	Member of zabbixGuests: proves group mapping gives different permissions (read-only guest)
Bind account	zabbix-search	Service account Zabbix uses to bind and search AD (see 3.9.1)
GPO	Zabbix gpo	Grants the gMSA access to the Zabbix Agent 2 folder on all DCs

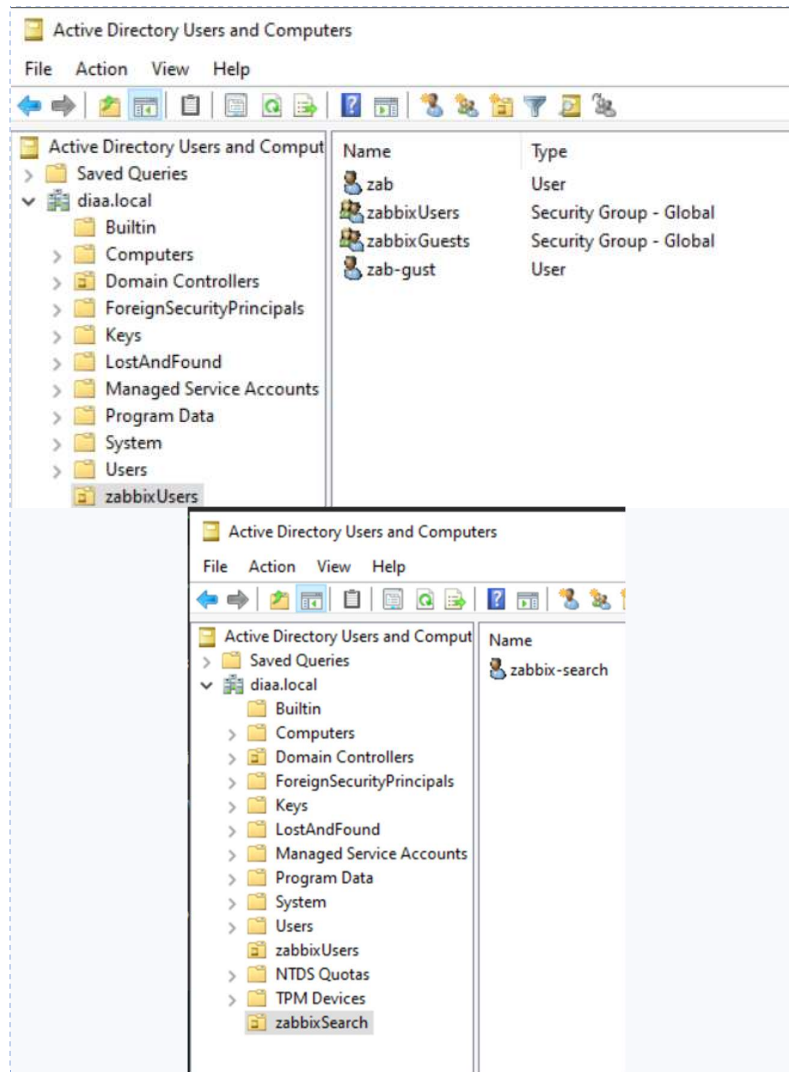


Figure 13: OU structure - zabbixUsers (users and groups) and zabbixSearch (bind account)

2.7 Additional Evidence

a) Replication health

repadmin /replsummary shows inbound and outbound replication between all domain controllers with no failures, and repadmin /showrepl lists each replication partner per naming context with the time and result of the last attempt.

```
C:\Users\azureadmin>repadmin /replsummary
Replication Summary Start Time: 2026-10-03 05:54:03

Beginning data collection for replication summary, this may take awhile:
.....

Source DSA          largest delta    fails/total  %%    error
ADC                 01m:40s        0 / 19      0
CAIROCHILD          01m:39s        0 / 5       0
GIZACHILD           01m:39s        0 / 5       0
pdc                 01m:39s        0 / 12      0

Destination DSA     largest delta    fails/total  %%    error
ADC                 01m:39s        0 / 12      0
CAIROCHILD          01m:38s        0 / 5       0
GIZACHILD           01m:39s        0 / 5       0
```

Figure 14: repadmin /replsummary output

```

C:\Users\azureadmin>repadmin /showrepl

Repadmin: running command /showrepl against full DC localhost
HQ\pdc
DSA Options: IS_GC
Site Options: (none)
DSA object GUID: 404c08b2-5ee7-4716-97dc-968157cc8620
DSA invocationID: 534bebf0-c494-44a8-a592-2f8a495bdcc6

===== INBOUND NEIGHBORS =====

DC=diaa,DC=local
HQ\ADC via RPC
DSA object GUID: 319352ef-3c1c-4fad-8460-b38c37221157
Last attempt @ 2026-10-03 05:54:21 was successful.

CN=Configuration,DC=diaa,DC=local
HQ\ADC via RPC
DSA object GUID: 319352ef-3c1c-4fad-8460-b38c37221157
Last attempt @ 2026-10-03 05:52:24 was successful.
nasrCity\CAIROCHILD via RPC
DSA object GUID: 5ba2928c-3e24-45aa-807e-00ffe01a49b2
Last attempt @ 2026-10-03 05:52:24 was successful.

CN=Schema,CN=Configuration,DC=diaa,DC=local
HQ\ADC via RPC
DSA object GUID: 319352ef-3c1c-4fad-8460-b38c37221157
Last attempt @ 2026-10-03 05:52:24 was successful.
nasrCity\CAIROCHILD via RPC
DSA object GUID: 5ba2928c-3e24-45aa-807e-00ffe01a49b2
Last attempt @ 2026-10-03 05:52:24 was successful.

DC=DomainDnsZones,DC=diaa,DC=local
HQ\ADC via RPC
DSA object GUID: 319352ef-3c1c-4fad-8460-b38c37221157
Last attempt @ 2026-10-03 05:52:24 was successful.

DC=ForestDnsZones,DC=diaa,DC=local
HQ\ADC via RPC
DSA object GUID: 319352ef-3c1c-4fad-8460-b38c37221157
Last attempt @ 2026-10-03 05:52:24 was successful.
nasrCity\CAIROCHILD via RPC
Last attempt @ 2026-10-03 05:52:24 was successful.

DC=cairo,DC=diaa,DC=local
HQ\ADC via RPC
DSA object GUID: 319352ef-3c1c-4fad-8460-b38c37221157
Last attempt @ 2026-10-03 05:52:24 was successful.
nasrCity\CAIROCHILD via RPC
DSA object GUID: 5ba2928c-3e24-45aa-807e-00ffe01a49b2
Last attempt @ 2026-10-03 05:52:24 was successful.

DC=giza,DC=diaa,DC=local
HQ\ADC via RPC

```

Figure 15: repadmin /showrepl output

b) RODC configuration

The RODC is controlled by a Password Replication Policy. The Allowed and Denied lists decide whose passwords may be cached, and the Advanced view lists the accounts whose passwords are actually cached on the RODC: only three objects, which shows that caching is limited to the accounts that need it.

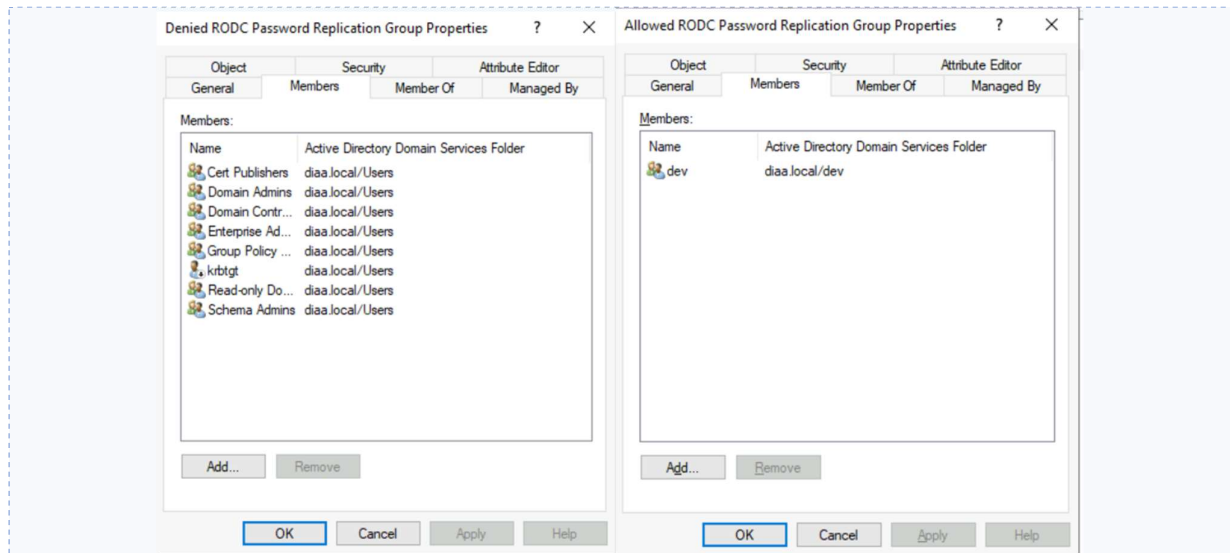


Figure 16: Password Replication Policy tab (Allowed / Denied RODC Groups)

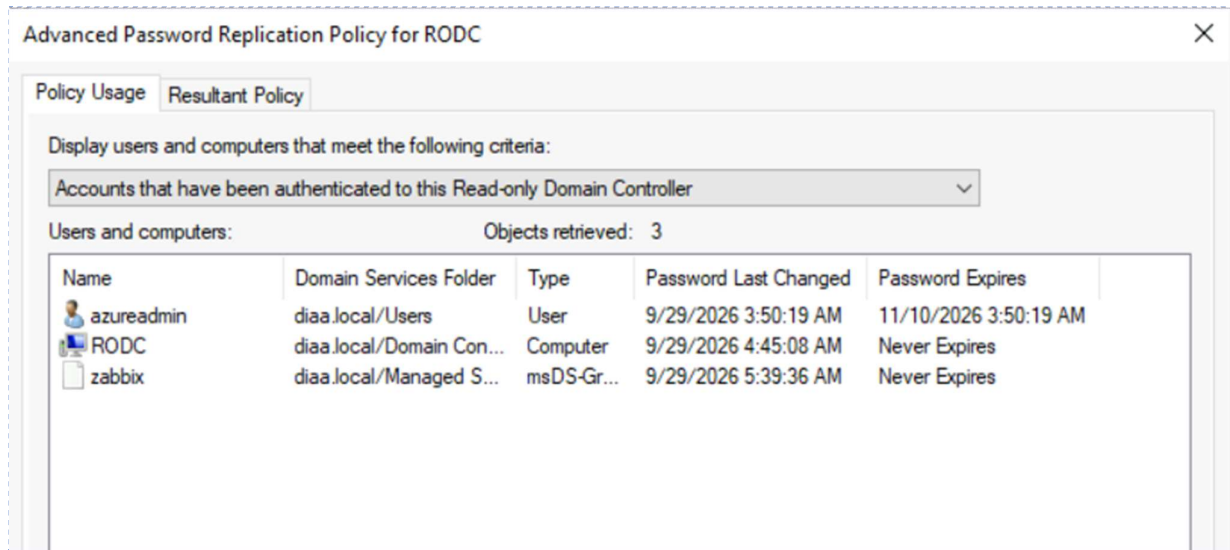


Figure 17: Advanced PRP — passwords cached on the RODC

3. Zabbix Monitoring

3.1 Zabbix Server Overview

Description

Zabbix 7.0 runs on a dedicated Linux server (zabbix-server, 10.1.7.20) in its own subnet. The five domain controllers are registered as hosts and linked to the Windows by Zabbix agent active template. The hosts list shows all of them available over the agent (ZBX) with about 127 items and 82 to 84 triggers each.

Item	Value
Zabbix version	7.0
Server IP / URL	10.1.7.20 (zabbix-server, subnet 10.1.7.0/24)
Database	[MySQL / PostgreSQL - confirm]
Monitored hosts	pdcc, ADC, RODC, CAIROCHILD, GIZACHILD (plus the Zabbix server itself)

Name	Items	Triggers	Graphs	Discovery	Web	Interface	Proxy	Templates	Status	Availability	Agent end
ADC	Items 128	Triggers 82	Graphs 18	Discovery 4	Web	10.1.3.20:10050		Windows by Zabbix agent active	Enabled	ZBX	None
CAIROCHILD	Items 128	Triggers 83	Graphs 18	Discovery 4	Web	10.1.5.20:10050		Windows by Zabbix agent active	Enabled	ZBX	None
GIZACHILD	Items 127	Triggers 82	Graphs 18	Discovery 4	Web	10.1.6.20:10050		Windows by Zabbix agent active	Enabled	ZBX	None
pdc	Items 128	Triggers 84	Graphs 18	Discovery 4	Web	10.1.2.20:10050		Windows by Zabbix agent active	Enabled	ZBX	None
RODC	Items 127	Triggers 82	Graphs 18	Discovery 4	Web	10.1.4.20:10050		Windows by Zabbix agent active	Enabled	ZBX	None
Zabbix server	Items 162	Triggers 84	Graphs 19	Discovery 6	Web	127.0.0.1:10050		Linux by Zabbix agent, Zabbix server health	Enabled	ZBX	None

Figure 20: Data collection > Hosts (all five DCs available over the agent)

3.2 Zabbix Agent 2 Configuration

Description

Zabbix Agent 2 is installed on every domain controller and works in active mode with the Windows by Zabbix agent active template. It reports to the Zabbix server at 10.1.7.20 and runs under a group Managed Service Account instead of LocalSystem, so the service has just the rights it needs, with no stored password.

Parameter	Value	Purpose
Server	10.1.7.20	Passive checks allowed from
ServerActive	10.1.7.20	Active checks / log items
Hostname	Same as the host name in Zabbix	Identity for active checks
Timeout	30	Check timeout in seconds
Service account	gMSA - zabbix\$	Runs the agent service

```
Timeout = 30
Server=10.1.7.20
ServerActive=10.1.7.20
```

Figure 21: zabbix_agent2.conf (key parameters)

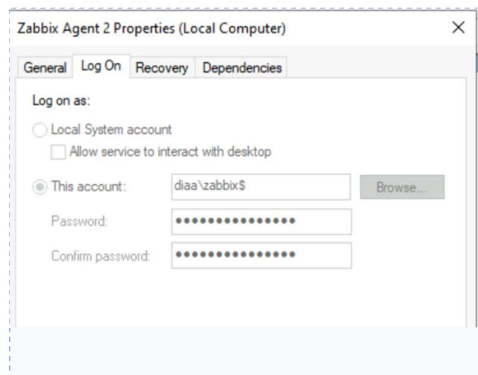


Figure 22: Agent service running as the gMSA

3.3 Service Account (gMSA) & Access

The agent runs under a Group Managed Service Account so no password is stored or rotated manually.

Component	Configuration
Security group	Zabbix-Agent-Servers - contains the pdc, adc, rodc, cairochild and gizachild computer accounts
gMSA	zabbix — password retrieval allowed for Zabbix-Agent-Servers
Domain groups	Performance Monitor Users, Event Log Readers
File system	Full control on C:\Program Files\Zabbix Agent 2
Service config	sc.exe config "Zabbix Agent 2" obj= diaa\zabbix\$ type= own

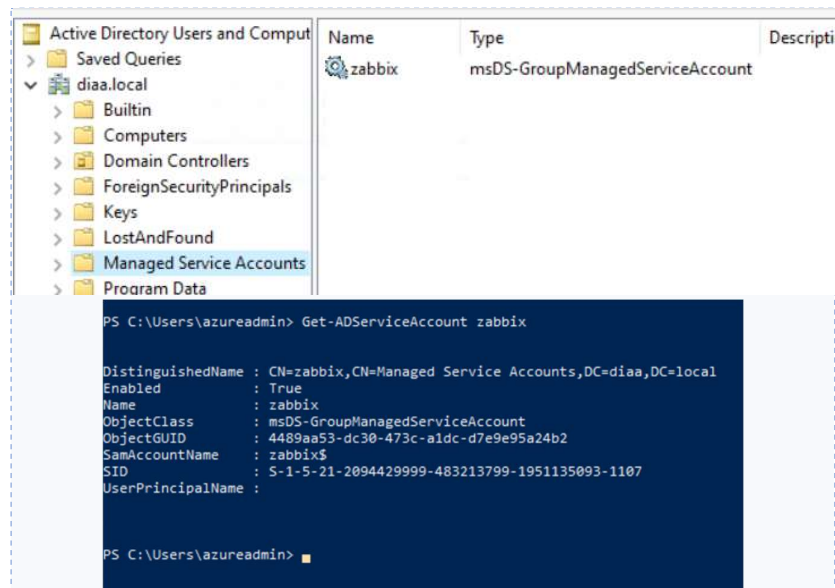


Figure 23: gMSA object and PrincipalsAllowedToRetrieveManagedPassword

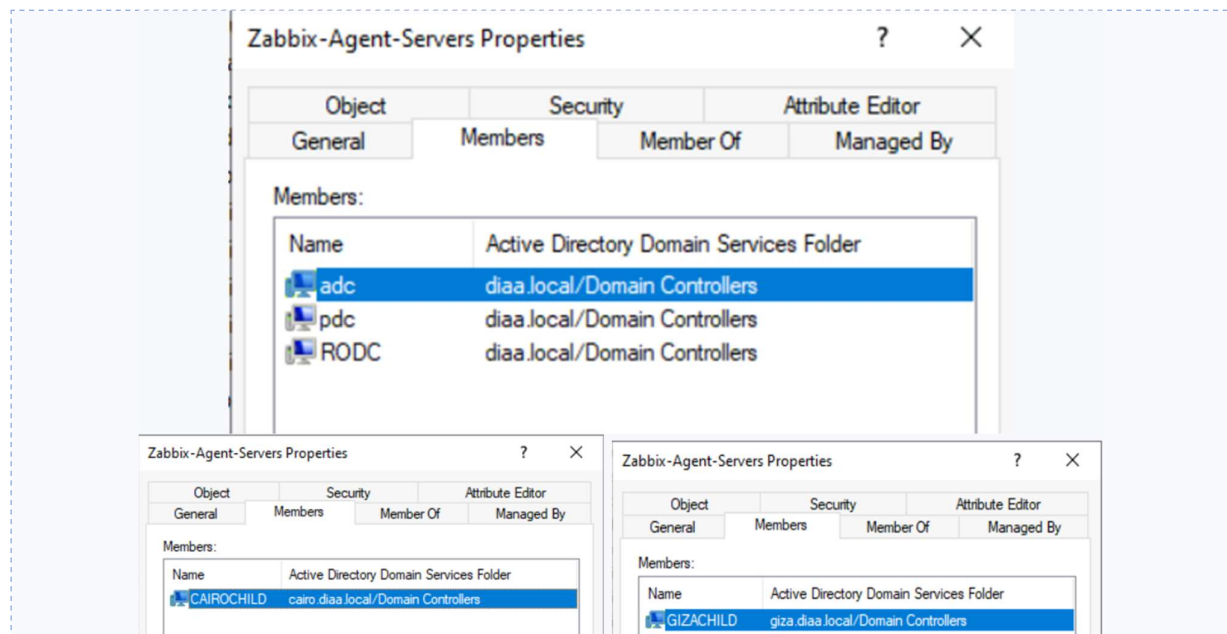


Figure 24: Zabbix-Agent-Servers group membership

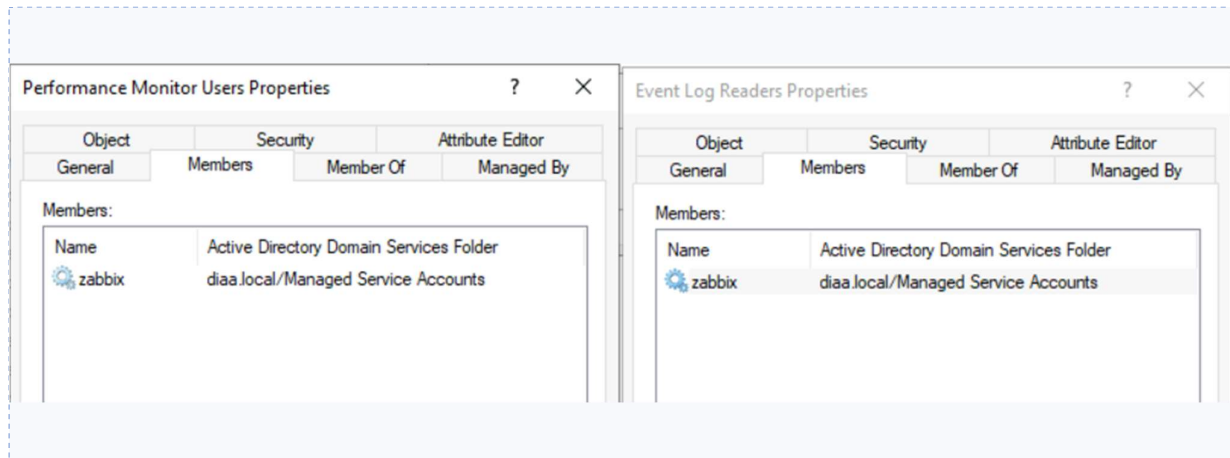


Figure 25: gMSA membership in Performance Monitor Users / Event Log Readers

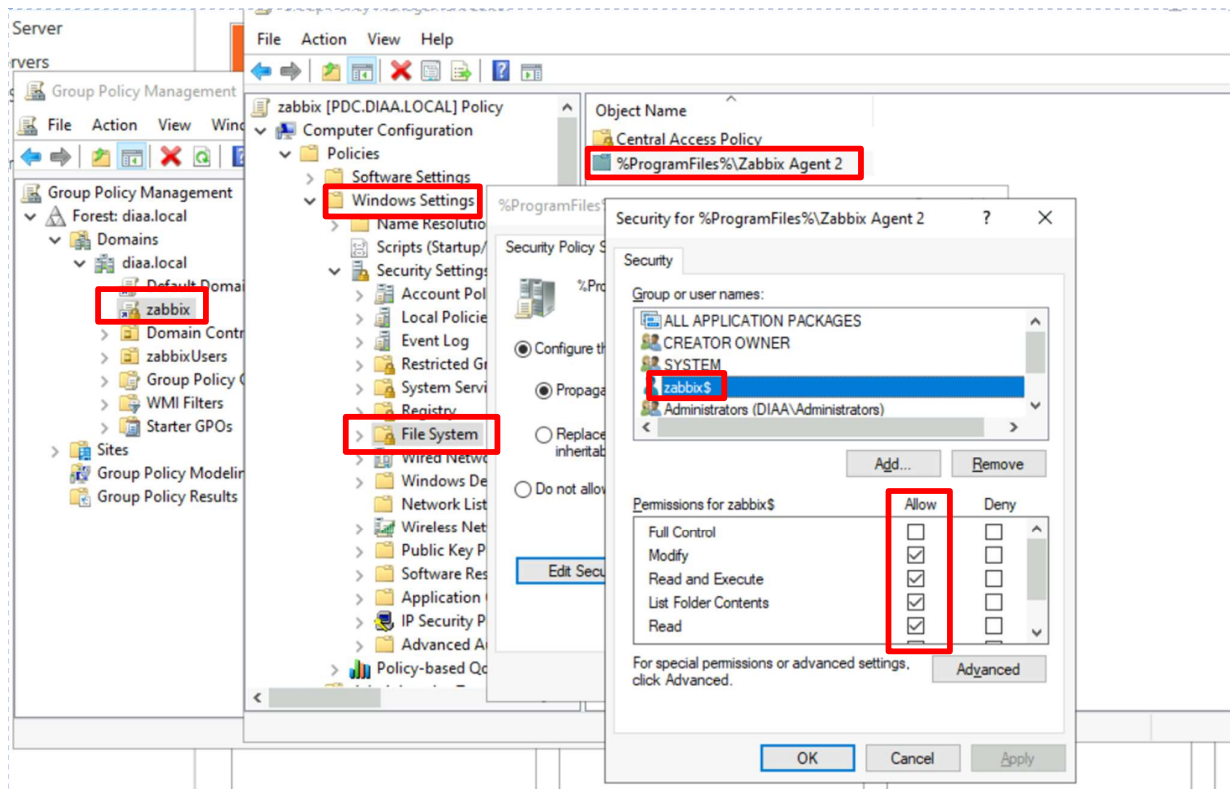


Figure 26: Folder permissions for the gMSA

3.4 Items

Description

On top of the standard template items, three items were added to the Windows by Zabbix agent active template so every domain controller inherits them: the Security event log, the AD DS replication status file, and the Zabbix agent's own log file. All three are Zabbix agent (active) items.

Item name	Key	Type	Interval
eventlog	eventlog[Security]	Zabbix agent (active)	3s

Item name	Key	Type	Interval
adds-replication	vfs.file.contents[C:\Program Files\Zabbix Agent 2\scripts\repl_status.txt]	Zabbix agent (active)	10s
zabbix agent logs	log["C:\Program Files\Zabbix Agent 2\zabbix_agent2.log"]	Zabbix agent (active)	3s
Template items	CPU, memory, disk, network and service items from the template	Zabbix agent (active)	Template default

Item

Item

Tags

Preprocessing

Parent items Windows by Zabbix agent active

Nameeventlog

TypeZabbix agent (active)

Keyeventlog[Security]

Type of informationLog

Update interval3s

Custom intervals

Type	Interval	Period	Action
Flexible	Scheduling	50s	1-7,00:00-24:00

Add

Remove

TimeoutGlobal

Override3s

Timeouts

HistoryDo not store

Store up to31d

Log time format

Description

Update

Clone

Execute now

Test

Clear history and tree

Figure 27: Item configuration - eventlog[Security]

Item

Item

Tags

Preprocessing

Nameadds-replication

TypeZabbix agent (active)

Keyvfs.file.contents[C:\Program Files\Zabbix Agent 2\scripts\repl_status.txt]

Select

Type of informationText

Update interval10s

Custom intervals

Type	Interval	Period	Action
Flexible	Scheduling	50s	1-7,00:00-24:00

Add

Remove

TimeoutGlobal

Override3s

Timeouts

HistoryDo not store

Store up to31d

Populates host inventory fieldNone-

Description

Update

Clone

Execute now

Test

Clear history and tree

Figure 28: Item configuration - adds-replication (repl_status.txt)

The screenshot shows the Zabbix Item configuration page for 'zabbix agent logs'. The 'Name' field is 'zabbix agent logs', 'Type' is 'Zabbix agent (active)', and 'Key' is 'log["C:\Program Files\Zabbix Agent 2\zabbix_agent2.log"]'. The 'Type of information' is 'Log' and the 'Update interval' is '3s'. There is a table for 'Custom intervals' with columns 'Type', 'Interval', 'Period', and 'Action'. It contains one row: 'Flexible', 'Scheduling', '50s', '1-7,00:00-24:00', and a 'Remove' button. Below this is an 'Add' button. The 'Timeout' section has 'Global', 'Override', '3s', and 'Timeouts' buttons. The 'History' section has 'Do not store', 'Store up to', and '31d' buttons. There are fields for 'Log time format' and 'Description'. At the bottom are buttons for 'Update', 'Clone', 'Execute now', 'Test', and 'Clear history'.

Figure 29: Item configuration - zabbix agent log

The adds-replication item reads repl_status.txt, a status file that a script on the PDC fills in with the result of the replication check. When failures are found, the file starts with ALERT and lists the failing partner, the last successful sync and the error code, and the Replication failure trigger (see 3.6) reads it.

```
# =====
# Script Name: repl.ps1 (Background Cached Version)
# =====
Import-Module ActiveDirectory -ErrorAction SilentlyContinue

$CacheFile = "C:\Program Files\Zabbix Agent 2\scripts\repl_status.txt"

try {
    $LocalDC = (Get-ADDomainController -Discover).Name
    $PartnerMetadata = Get-ADReplicationPartnerMetadata -Target $LocalDC -ErrorAction Stop
    $FailedLinks = ($PartnerMetadata | Where-Object { $_.LastReplicationResult -ne 0 })

    if ($FailedLinks.Count -eq 0) {
        "Healthy" | Out-File -FilePath $CacheFile -Force
    } else {
        $Report = [System.Collections.Generic.List[string]]::new()
        $Report.Add("ALERT: Found $($FailedLinks.Count) replication failure(s) on DC [$LocalDC]")
        $Report.Add("-----")

        foreach ($Link in $FailedLinks) {
            $Report.Add("Partner DC : $($Link.Partner.Split(',').Replace('CN=', ''))")
            $Report.Add("Partition : $($Link.NamingContext)")
            $Report.Add("Last Sync : $($Link.LastReplicationSuccess)")
            $Report.Add("Error Code : $($Link.LastReplicationResult)")
            $Report.Add("-----")
        }

        $Report | Out-File -FilePath $CacheFile -Force
    }
} catch {
    "CRITICAL SCRIPT ERROR: $_" | Out-File -FilePath $CacheFile -Force
}

# =====
# Script Name: schaduler.ps1
# =====

$Action = New-ScheduledTaskAction -Execute "powershell.exe" -Argument "-NoProfile -NonInteractive -ExecutionPolicy Bypass -File 'C:\Program Files\Zabbix Agent 2\scripts\repl.ps1'"
$Trigger = New-ScheduledTaskTrigger -Once -At (Get-Date) -RepetitionInterval (New-TimeSpan -Minutes 5)
$Principal = New-ScheduledTaskPrincipal -UserId "NT AUTHORITY\SYSTEM" -LogonType ServiceAccount -RunLevel Highest
Register-ScheduledTask -TaskName "Zabbix_AD_Replication_Check" -Action $Action -Trigger $Trigger -Principal $Principal
```

Figure 30: Replication check script and its schedule

3.5 Discovery Rules

Description

A network discovery rule named domain controllers scans 10.1.0.0/16 every minute with a Zabbix agent system.hostname check. Every host that answers is picked up by a discovery action that adds the host, puts it in the domain controllers host group, links the Windows by Zabbix agent active template and sends a Telegram message. This is how all five domain controllers appeared in Zabbix without creating hosts by hand.

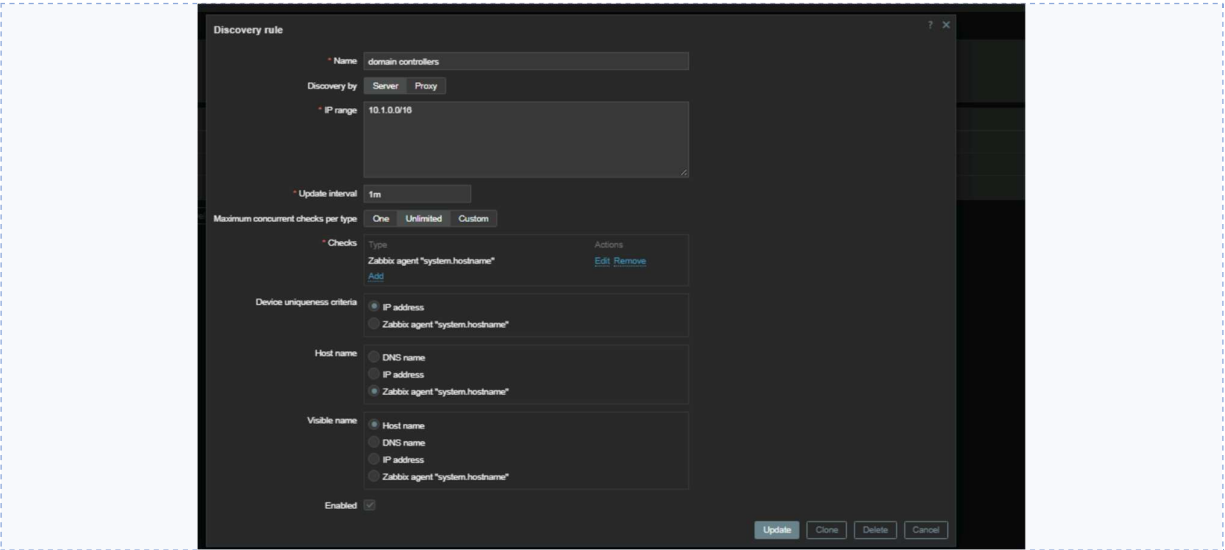


Figure 31: Network discovery rule - domain controllers (10.1.0.0/16)

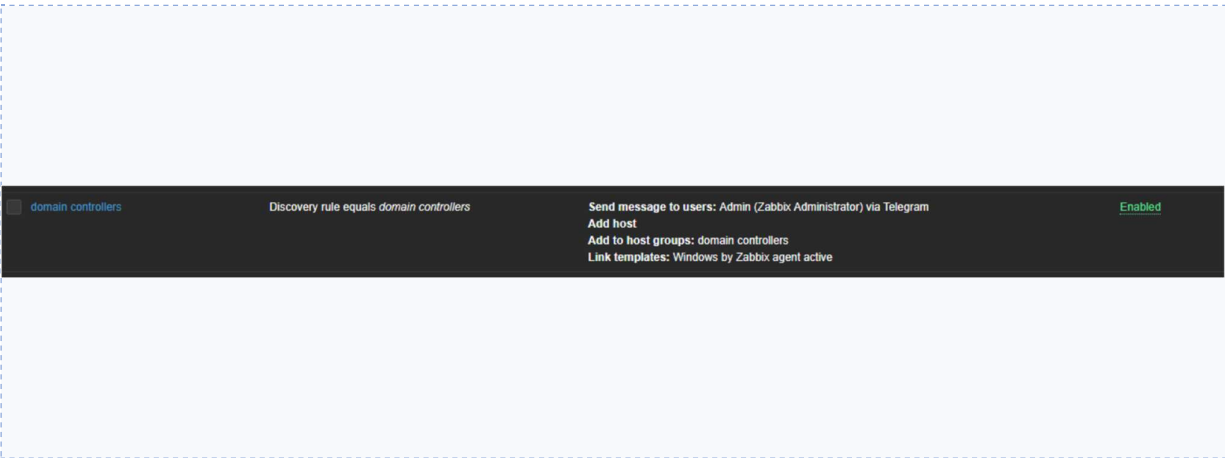


Figure 32: Discovery action - add host, add to group, link template, notify

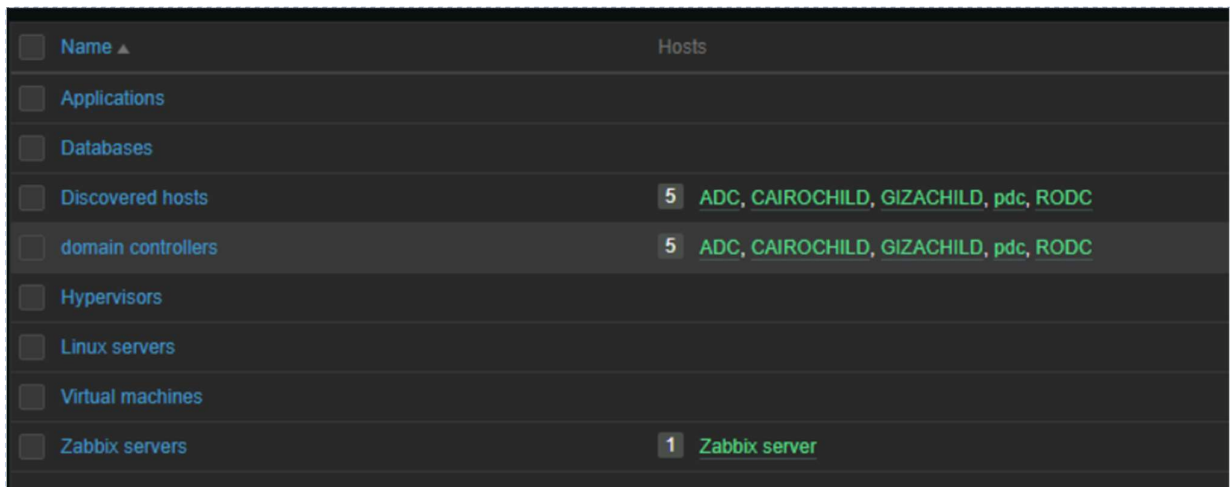


Figure 33: Host groups - the five DCs in domain controllers

3.6 Triggers

Alerting is driven by triggers. Replication failure (Disaster) is a custom trigger on the adds-replication item of the PDC. The template's own triggers, such as agent availability and CPU queue length, use the same notification path.

Trigger name	Expression	Severity
Replication failure	find(/pdc/vfs.file.contents[C:\Program Files\Zabbix Agent 2\scripts\repl_status.txt],#1,"like","ALERT")=1	Disaster
Windows: Zabbix agent is not available (or nodata for {\$AGENT.NODATA_TIMEOUT})	Template trigger: fires when the active agent stops reporting	High
Windows: CPU queue length is too high (over 3 for 5m)	Template trigger on the CPU queue length item	Warning

The screenshot shows the Zabbix Trigger configuration page. The 'Trigger' tab is active, showing the configuration for a trigger named 'Replication failure'. The 'Event name' is also 'Replication failure'. The 'Operational data' field is empty. The 'Severity' is set to 'Disaster'. The 'Expression' is a Zabbix user-defined function: `find(/pdc/vfs.file.contents[C:\Program Files\Zabbix Agent 2\scripts\repl_status.txt],#1,"like","ALERT")=1`. Below the expression field is a link to the 'Expression constructor'. The 'OK event generation' is set to 'Expression'. The 'PROBLEM event generation mode' is set to 'Single'. The 'OK event closes' is set to 'All problems'. The 'Allow manual close' checkbox is unchecked. The 'Menu entry name' is 'Trigger URL' and the 'Menu entry URL' is empty.

Trigger

Trigger Tags Dependencies

* Name Replication failure

Event name Replication failure

Operational data

Severity Not classified Information Warning Average High **Disaster**

* Expression `find(/pdc/vfs.file.contents[C:\Program Files\Zabbix Agent 2\scripts\repl_status.txt],#1,"like","ALERT")=1` Add

[Expression constructor](#)

OK event generation Expression Recovery expression None

PROBLEM event generation mode Single Multiple

OK event closes All problems All problems if tag values match

Allow manual close ☐

Menu entry name ? Trigger URL

Menu entry URL

Figure 34: Trigger configuration - Replication failure (expression and severity)

3.7 Telegram Notifications

Description

Alerts are delivered to a Telegram group. A webhook media type (Telegram) uses a bot to post to the group chat, and the Admin user has a Telegram media entry. Three trigger actions send the notifications: agent not sending data, replication failure, and report problems to Zabbix administrators. The last figure shows real alerts received in the Zabbix alerts group.

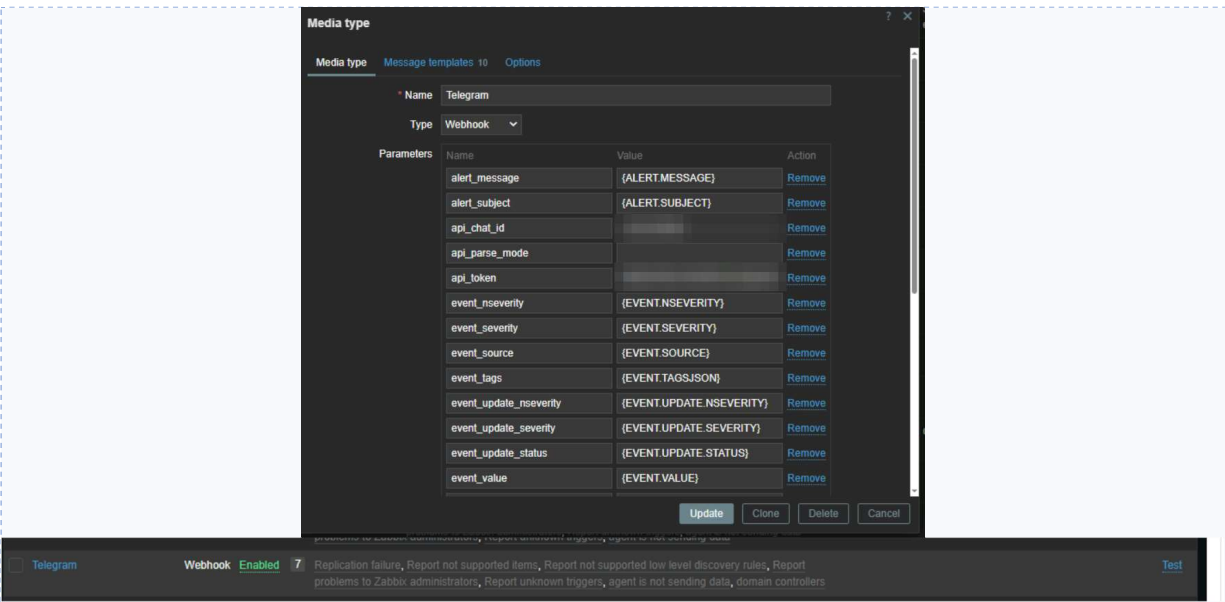


Figure 35: Media type - Telegram webhook (bot token and chat ID blurred)

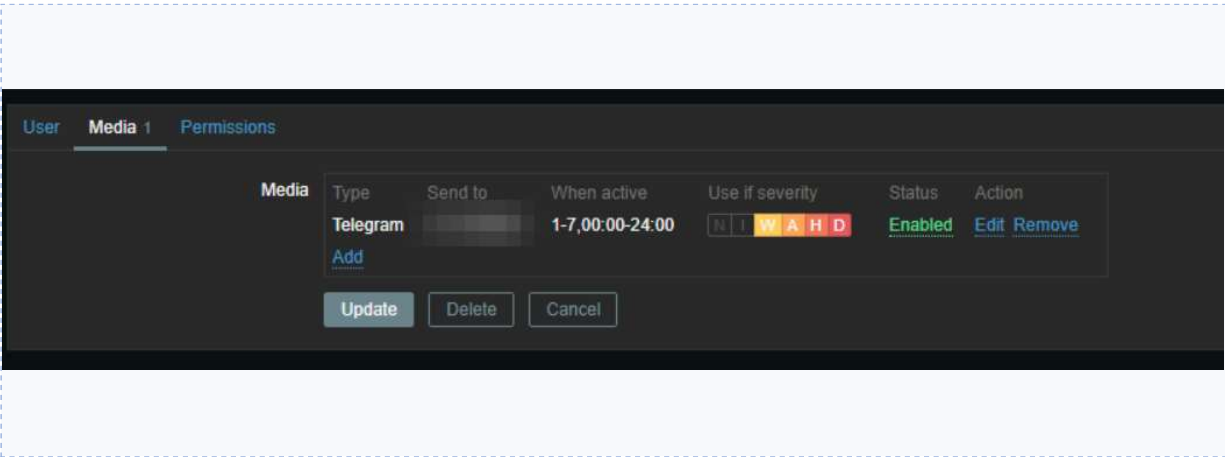


Figure 36: User media - Telegram

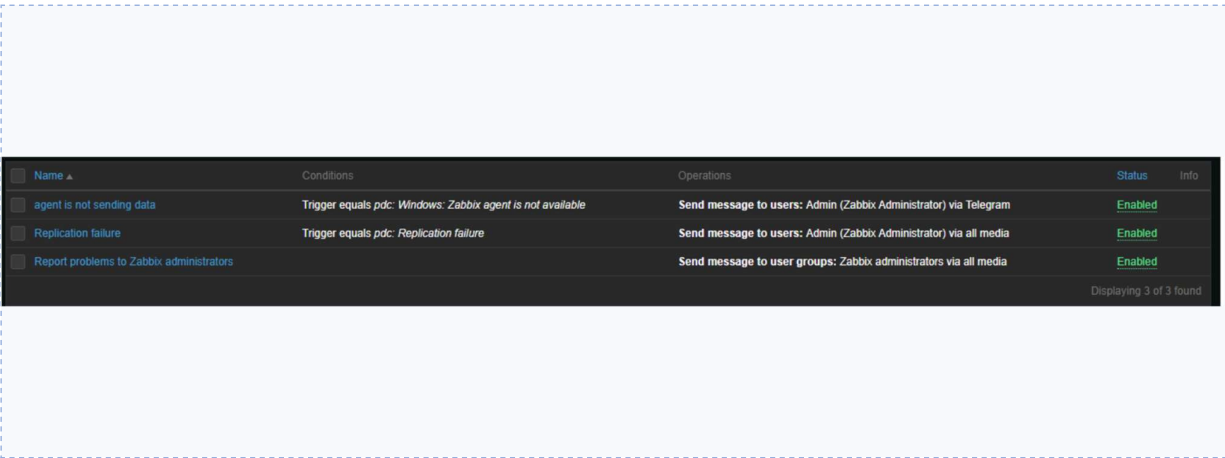


Figure 37: Trigger actions - agent availability, replication failure, report problems

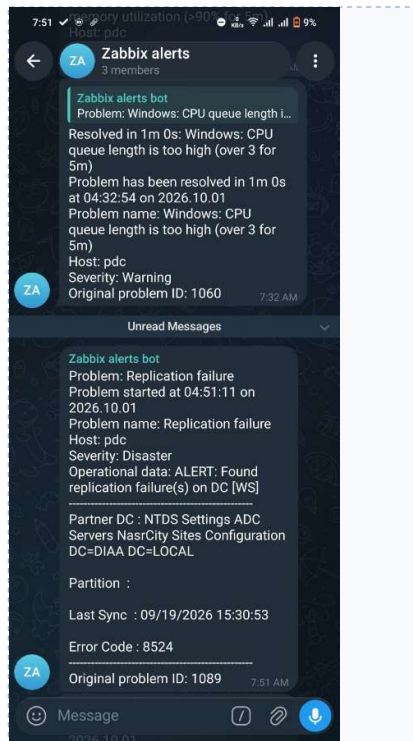


Figure 38: Telegram group receiving the alerts

3.8 Dashboard

Description

A dashboard gathers what matters for the domain controllers on three tabs: Performance (CPU, memory and disk per DC), Logs (Windows Security event log entries) and Agent Logs (the agent log files), so the health of the whole forest can be read in one place.

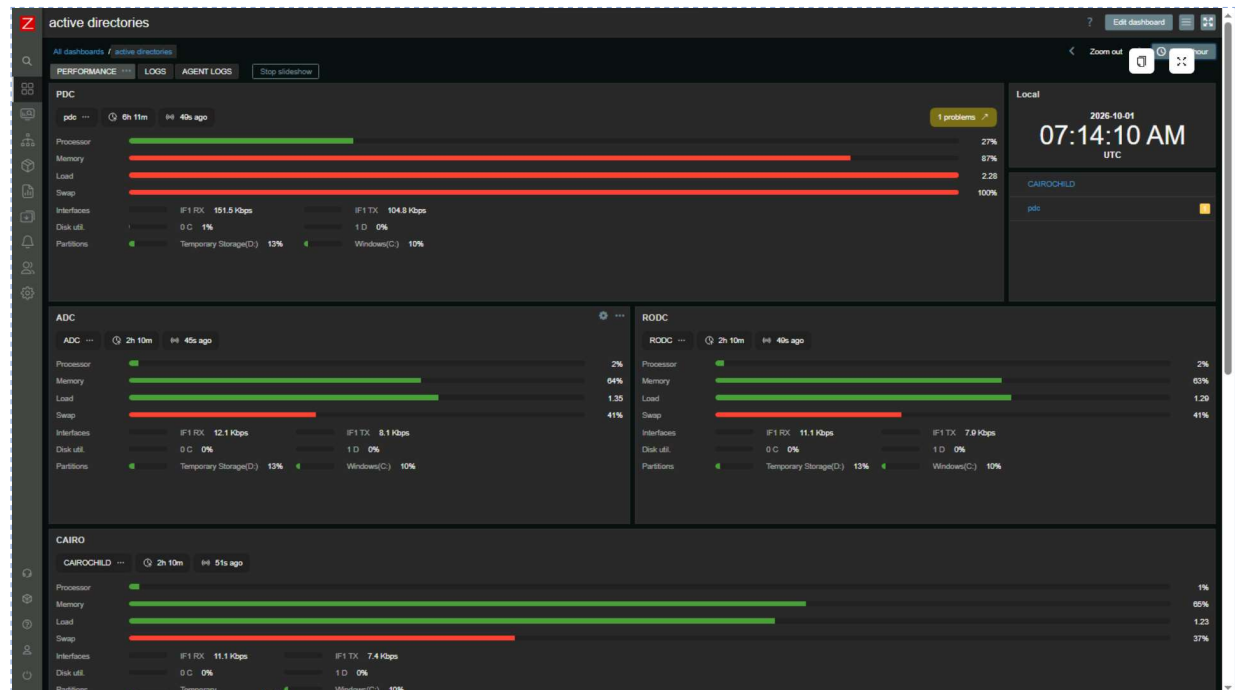


Figure 39: Dashboard — Performance tab

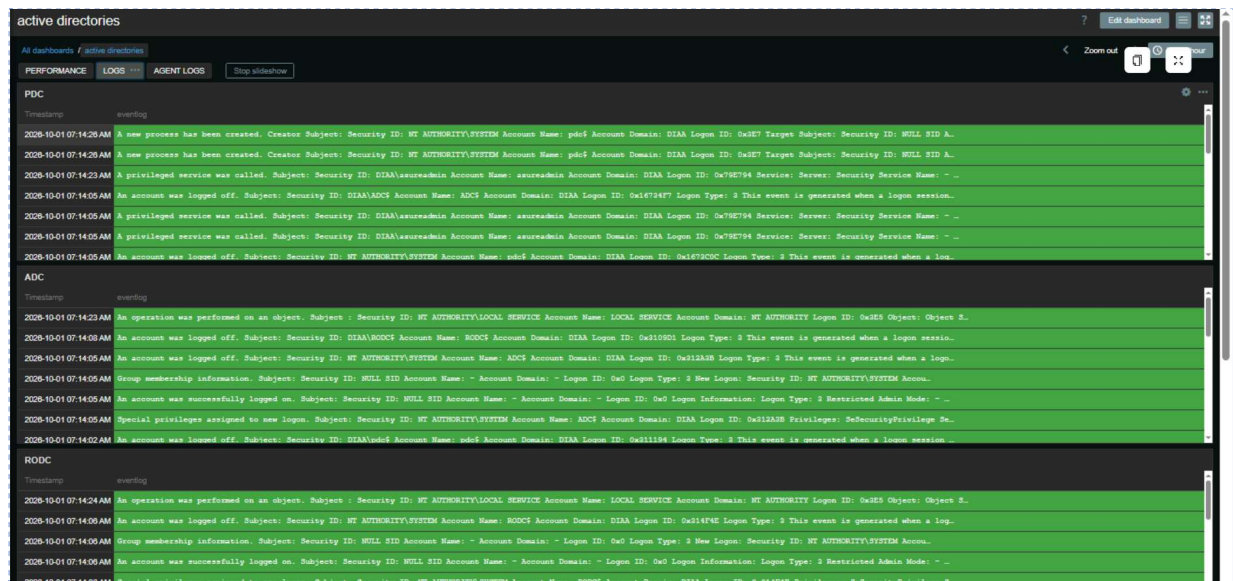


Figure 40: Dashboard — Logs tab

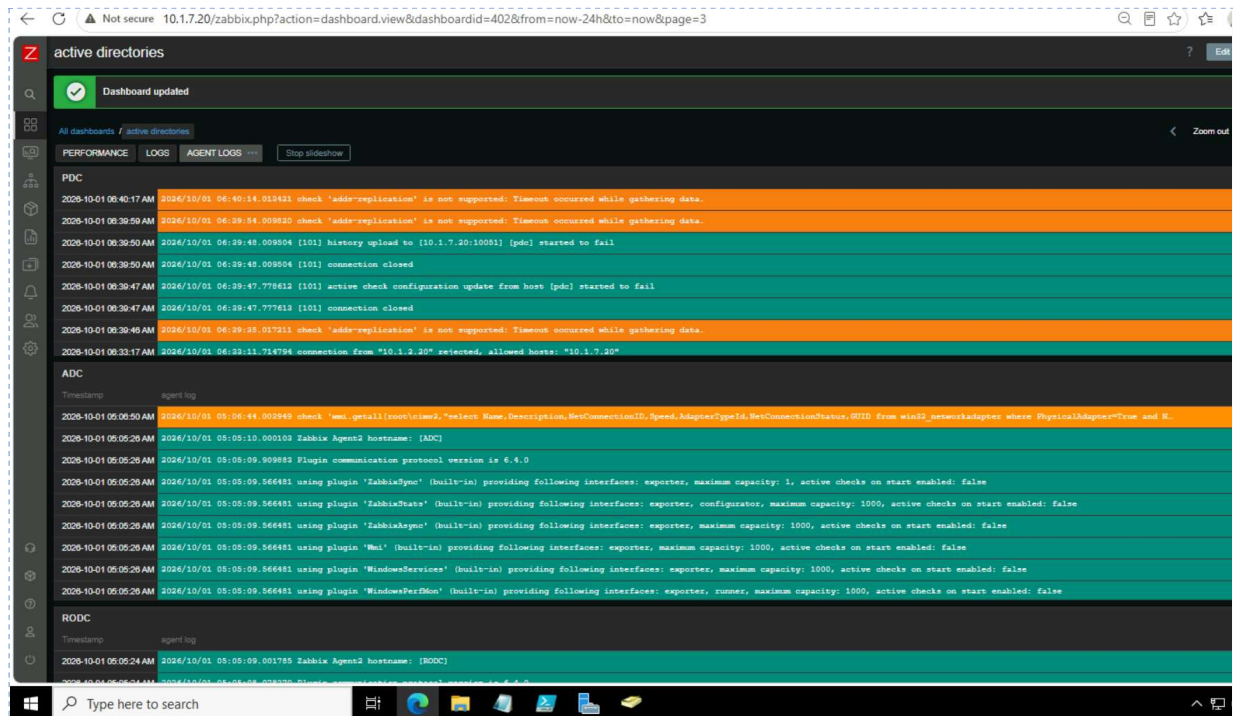


Figure 41: Dashboard — Agent Logs tab

3.9 LDAP Authentication & JIT Users

Description

Zabbix users sign in with their Active Directory accounts. The server uses a bind account to search the zabbixUsers OU, and JIT provisioning creates the Zabbix user at the first successful login and assigns the user group and role from the AD group: zabbixUsers maps to Zabbix administrators (Super admin role) and zabbixGuests maps to Guests (Guest role).

Setting	Value
LDAP server	10.1.2.20 (pdc), port 389
Base DN / search attribute	OU=zabbixUsers,DC=DIAA,DC=LOCAL / sAMAccountName

Setting	Value
Bind DN	CN=zabbix-search,OU=zabbixSearch,DC=DIAA,DC=LOCAL
JIT provisioning	Enabled - group mapping via memberOf
Deprovisioned users group	Disabled
Default authentication	LDAP - local accounts denied

LDAP Server

* Name

diaa.local

* Host

10.1.2.20

* Port

389

* Base DN

OU=zabbixUsers,DC=DIAA,DC=LOCAL

* Search attribute

sAMAccountName

Bind DN

CN=zabbix-search,OU=zabbixSearch,DC=DIAA,DC=LOCAL

Bind password

Change password

Description

Figure 42: Authentication → LDAP settings

LDAP Server

Configure JIT provisioning

☒

Group configuration ?

memberOfgroupOfNames

Group name attribute

CN

User group membership attribute

memberof

User name attribute

givenName

User last name attribute

sn

* User group mapping

LDAP group pattern	User groups	User role	Action
zabbixUsers	Zabbix administrators	Super admin role	Remove
zabbixGuests	Guests	Guest role	Remove
Add			

Media type mapping ?

Name	Media type	Attribute	Action
Add			

Advanced configuration

Figure 43: Group mapping (memberOf → user group / role)

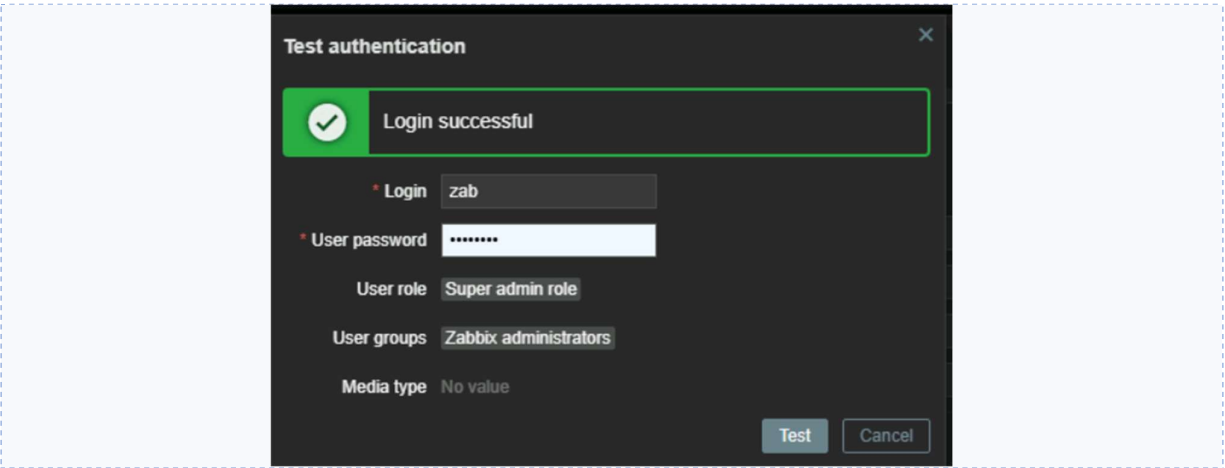


Figure 44: Successful LDAP login

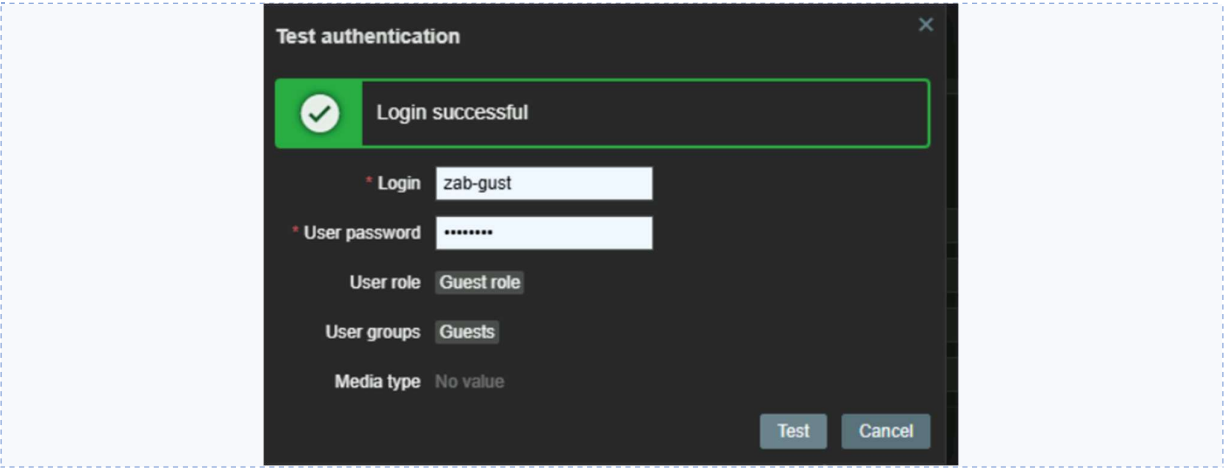


Figure 45: Second test user: different group, different Zabbix role

Users							
☐	Username ▲	Name	Last name	User role	Groups	Is online?	Login Frontend access
☐	guest			Guest role	Guests, Internal	No	Ok LDAP
☐	zab-gust	zab-gust		Guest role	Guests	Yes (2026-10-03 05:03:38 AM)	Ok LDAP

Figure 46: Users created automatically by JIT (frontend access: LDAP)

3.9.1 LDAP Bind Account vs gMSA

The LDAP bind uses a regular service account rather than a gMSA. Zabbix runs on Linux and its LDAP settings need a fixed bind DN and password: a gMSA password is retrieved and rotated by Windows for domain-joined servers, so it cannot be used here.

	Agent account	LDAP bind account
Type	gMSA (zabbix\$)	Regular service account
Used by	Zabbix Agent 2 Windows service	Zabbix server (Linux) LDAP search
Password	Auto-rotated by AD	Static, long and random, no expiry
Why this type	Windows service on domain-joined DCs	Zabbix needs a fixed password in its config

Hardening applied to the bind account:

- Least privilege: plain Domain Users member, no admin rights; default directory read access is enough for search
- Password: long and random, set to never expire and user cannot change it, so Zabbix is not locked out by expiry
- Placement: the account lives in its own zabbixSearch OU, separate from the zabbixUsers OU that Zabbix searches
- Logon restrictions: interactive and remote logon denied, so the account can only be used to bind
- Transport: plain LDAP on port 389 in this lab, so the bind password crosses the VNet in cleartext. LDAPS on port 636, with a certificate on the DC, is the recommended improvement

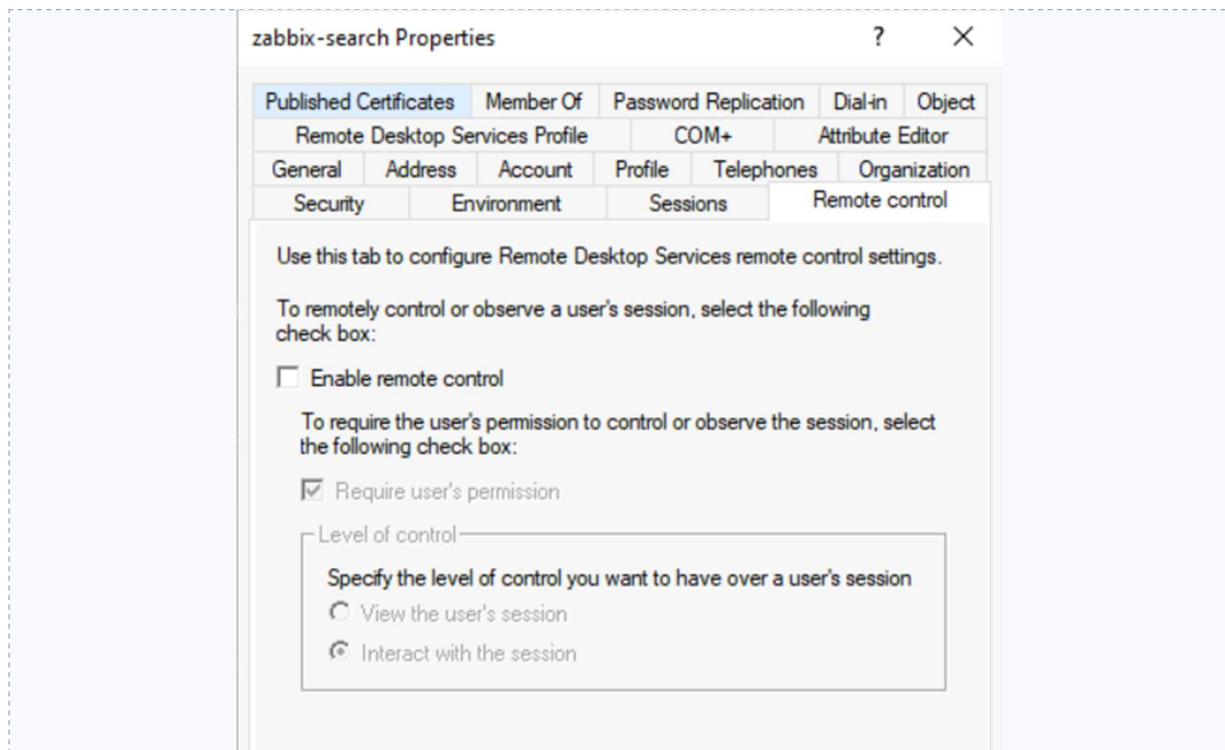
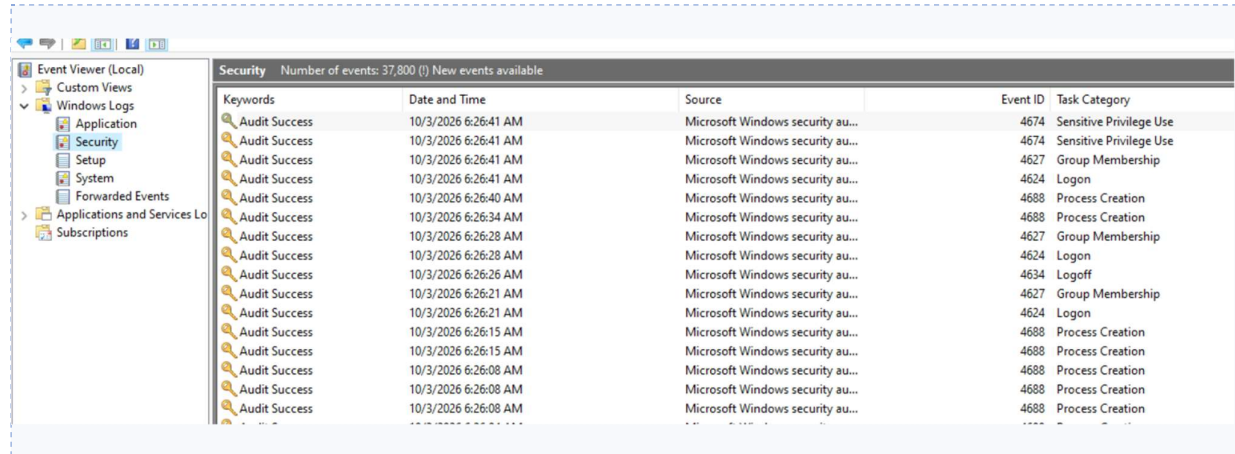


Figure 47: Bind account (zabbix-search) properties

3.10 Logs

Description

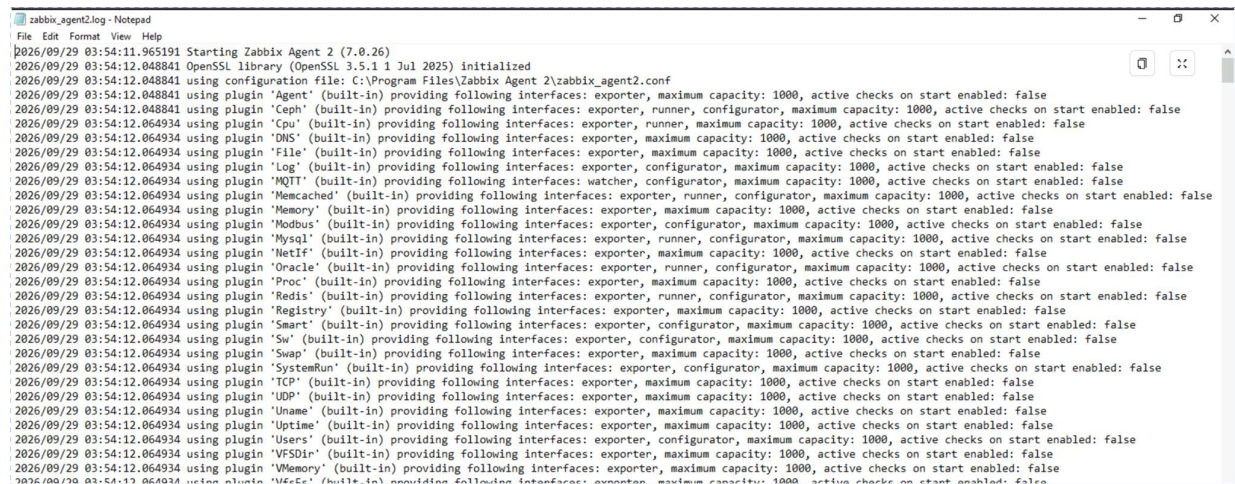
Windows Security event logs and the Zabbix agent log files are collected into Zabbix as log items, so authentication activity and Zabbix's own errors can be reviewed in the same place as the metrics. The first figure shows Security event log entries received from a domain controller, the second the agent log file contents.



The screenshot shows the Windows Event Viewer interface. The left pane displays the 'Event Viewer (Local)' tree with 'Security' selected under 'Windows Logs'. The right pane shows a list of security events. The table below represents the data visible in the right pane.

Keywords	Date and Time	Source	Event ID	Task Category
Audit Success	10/3/2026 6:26:41 AM	Microsoft Windows security au...	4674	Sensitive Privilege Use
Audit Success	10/3/2026 6:26:41 AM	Microsoft Windows security au...	4674	Sensitive Privilege Use
Audit Success	10/3/2026 6:26:41 AM	Microsoft Windows security au...	4627	Group Membership
Audit Success	10/3/2026 6:26:41 AM	Microsoft Windows security au...	4624	Login
Audit Success	10/3/2026 6:26:40 AM	Microsoft Windows security au...	4688	Process Creation
Audit Success	10/3/2026 6:26:34 AM	Microsoft Windows security au...	4688	Process Creation
Audit Success	10/3/2026 6:26:28 AM	Microsoft Windows security au...	4627	Group Membership
Audit Success	10/3/2026 6:26:28 AM	Microsoft Windows security au...	4624	Login
Audit Success	10/3/2026 6:26:26 AM	Microsoft Windows security au...	4634	Logoff
Audit Success	10/3/2026 6:26:21 AM	Microsoft Windows security au...	4627	Group Membership
Audit Success	10/3/2026 6:26:21 AM	Microsoft Windows security au...	4624	Login
Audit Success	10/3/2026 6:26:15 AM	Microsoft Windows security au...	4688	Process Creation
Audit Success	10/3/2026 6:26:15 AM	Microsoft Windows security au...	4688	Process Creation
Audit Success	10/3/2026 6:26:08 AM	Microsoft Windows security au...	4688	Process Creation
Audit Success	10/3/2026 6:26:08 AM	Microsoft Windows security au...	4688	Process Creation
Audit Success	10/3/2026 6:26:08 AM	Microsoft Windows security au...	4688	Process Creation

Figure 48: Windows Security event log entries collected by Zabbix



The screenshot shows a Notepad window displaying the contents of the 'zabbix_agent2.log' file. The log entries show the initialization of the Zabbix Agent 2 (7.0.26) and the loading of various modules. Each module entry includes its name, type (built-in), and a list of interfaces it provides with their respective maximum capacities and whether active checks are enabled on start.

```

2026/09/29 03:54:11.965191 Starting Zabbix Agent 2 (7.0.26)
2026/09/29 03:54:12.048841 OpenSSL library (OpenSSL 3.5.1 1 Jul 2025) initialized
2026/09/29 03:54:12.048841 using configuration file: C:\Program Files\Zabbix Agent 2\zabbix_agent2.conf
2026/09/29 03:54:12.048841 using plugin 'Agent' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.048841 using plugin 'Ceph' (built-in) providing following interfaces: exporter, runner, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Cpu' (built-in) providing following interfaces: exporter, runner, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Dns' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'File' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Log' (built-in) providing following interfaces: exporter, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'MQTT' (built-in) providing following interfaces: watcher, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Memcached' (built-in) providing following interfaces: exporter, runner, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Memory' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Modbus' (built-in) providing following interfaces: exporter, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Mysql' (built-in) providing following interfaces: exporter, runner, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'NetIf' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Oracle' (built-in) providing following interfaces: exporter, runner, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Proc' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Redis' (built-in) providing following interfaces: exporter, runner, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Registry' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Smart' (built-in) providing following interfaces: exporter, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Sw' (built-in) providing following interfaces: exporter, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Swap' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'SystemRun' (built-in) providing following interfaces: exporter, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'TCP' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'UDP' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Uname' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Uptime' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'Users' (built-in) providing following interfaces: exporter, configurator, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'VFSDir' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'VFSFile' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false
2026/09/29 03:54:12.064934 using plugin 'VFSFile' (built-in) providing following interfaces: exporter, maximum capacity: 1000, active checks on start enabled: false

```

Figure 49: Zabbix agent log file contents (log item)

4. Testing & Validation

Test	Expected result	Actual result	Pass?
Stop replication / DC offline	Zabbix trigger fires + Telegram alert	Replication failure (Disaster) raised for the PDC and delivered to the Telegram group (Figure 38)	Pass
Domain user logs in to Zabbix	JIT account created	Test login succeeded with the AD password (Figure 44); the JIT-created user is listed with LDAP access (Figure 46)	Pass
Local Admin login attempt	Denied (LDAP-only)	Pending - try the local Admin login and add a screenshot	Pass
RODC password caching	Only allowed accounts cached	Advanced PRP lists only 3 cached objects (Figure 17)	Pass
Login as user 1 vs user 2	Different Zabbix roles via group mapping	zab: Super admin role, Zabbix administrators. zab-gust: Guest role, Guests (Figures 44, 45)	Pass
FSMO transfer to ADC	netdom shows Infrastructure Master on ADC	Infrastructure master is adc.diaa.local after the move (Figure 11)	Pass

5. Troubleshooting Log

Issue	Cause	Fix
Access Denied installing gMSA on a DC	Stale Kerberos ticket cache for SYSTEM	klist purge -li 0x3e7, then reinstall the gMSA
Agent service failed to start under gMSA	No permission on the agent folder	Granted the gMSA control on C:\Program Files\Zabbix Agent 2
eventlog[Security] Not supported	Item type / ServerActive not applied	Set item type to Zabbix agent (active) and fix ServerActive in the agent config

6. Conclusion & Lessons Learned

This project built a three-domain Active Directory forest across five Azure subnets and four AD sites, and showed that it can be operated from one place. Replication health is checked continuously, the state and event logs of every domain controller are collected by Zabbix, and failures reach the administrators through Telegram. Access to Zabbix itself is tied to Active Directory groups, so permissions follow group membership.

The hardest parts were running Zabbix Agent 2 under a gMSA on domain controllers and getting the active checks to work. A stale Kerberos ticket cache for SYSTEM blocked the gMSA install, the service needed explicit permissions on its folder, and eventlog items stayed unsupported until the item type and ServerActive were corrected. AD replication health also has no built-in item, so a status file written by a script and read by the agent was used.

Possible improvements:

- Use LDAPS instead of plain LDAP for the Zabbix bind, so the bind password is encrypted.
- Extend the Terraform deployment with configuration management (Ansible or DSC) for AD promotion, gMSA, GPO and the Zabbix configuration, so the whole lab can be rebuilt unattended.
- Add a second alerting channel such as email, and monitor DNS and the RODC-specific services.